

Universidade Leiria • Oeste

Escola de Tecnologia e Gestão

Desenvolvimento de Aplicações Distribuídas

Web Application Models

Web Applications, Main Model Categories; Rendering Strategies; SPA Frameworks; Mobile and Multiplatform

Marco Monteiro and Ricardo Gomes

Summary

1. Web Applications
2. Web Application Models
   - Multi-Page Applications
   - Single-Page Applications
   - Hybrid Applications
3. Rendering Strategies
4. SPA Frameworks
5. Mobile and Multiplatform

Web Applications

Web Application Definition

- A web application is a software system that:
  - Is delivered over the Web
  - Is accessed through a browser or similar client
  - Uses HTTP and other web protocols (e.g., HTTPS, WebSocket, QUIC)
  - Provides dynamic interaction beyond static content
    - Unlike a static website, a web application is interactive: the server and client exchange requests and responses, and the application can adapt dynamically to user input or system events
  - Appears to the user as one cohesive system
    - Whether it consists of multiple pages (e.g., a banking portal) or a single page (e.g., Gmail), a consistent visual style and shared navigation, state, logic, and user sessions enable it to function as a coherent, unified system for the end-user

HTTP Protocol

- Request/Response Model
  - Communication occurs as a pair: the client sends a request, and the server replies with a response.
- Text-Based (Human-Readable)
  - Especially in HTTP/1.x, messages are plain text. Later versions (HTTP/2, HTTP/3) introduced binary framing but retained logical compatibility.
- Stateless
  - Each request is independent; the server does not retain client state between requests unless mechanisms like cookies, sessions, or tokens are used.
- Client-Server Architecture
  - Clear separation of roles: the client initiates requests; the server responds with resources or services
- Extensible via Headers
  - HTTP headers allow extensibility for caching, authentication, compression, etc.

HTTP Protocol

- Media-Type Agnostic
  - Supports transfer of any type of content, indicated via MIME types (Content-Type).
- Uniform Resource Identification
  - Uses URL/URI to identify resources consistently across the Web.
- Stateless but Cacheable
  - Responses can be cached to improve efficiency.
- Transport Independent
  - Usually runs over TCP (HTTP/1–2) or QUIC (HTTP/3) but not strictly tied to one transport protocol.
- Versions (with backward compatibility)
  - HTTP/1.0 - one request per TCP connection.
  - HTTP/1.1 - persistent connections (keep-alive).
  - HTTP/2 - multiplexed streams.
  - HTTP/3 - built over QUIC (UDP).

Web Application Models

Multi-Page Applications

Single-Page Applications

Hybrid Applications

Web Application Model - Definition

- A **web application model** is an abstraction that describes how a web app is structured, executed, and delivered to users. It defines:
  - Client-side responsibilities: presentation, interaction, and partial logic.
  - Server-side responsibilities: data storage, business logic, authentication, orchestration.
  - Communication model: HTTP requests/responses, WebSockets, RPC, GraphQL, etc.
- A model determines page **structure**, **navigation** style, client-server **interaction**, and where **rendering** occurs, which may be primarily on the server, primarily on the client, or a combination of both. Different models balance these elements differently, giving rise to distinct categories.

Web Application Model - Main Categories

- Multi-Page Applications (MPA)
  - Multiple pages, full-page reloads
  - Primarily **SSR** (server-side rendering)
- Single-Page Applications (SPA)
  - Single page, dynamic updates without full reloads
  - Primarily **CSR** (client-side rendering)
- Hybrid / Mixed Applications
  - Multiple pages with partial client-side updates
  - Combination of **SSR** and **CSR**

Multi-Page Applications (MPA)

- Structure:
  - Multiple pages, each with its own URL and HTML content
- Navigation Style:
  - Navigation triggers full-page reloads
  - Server generates each page independently
- Client-Server Interaction:
  - Server handles business logic, rendering, and state
  - Client mostly receives rendered HTML and applies small JS for UI enhancements
- Rendering:
  - Primarily server-side rendering (SSR)
  - Minimal client-side updates
- Examples:
  - Apps: Banking portals, Wikipedia, classic e-commerce sites
  - Frameworks: Laravel, Django, Spring MVC, ASP.NET MVC

Multi-Page Applications (MPA)

- Key Features / Benefits:
  - Strong SEO and reliable analytics (fully rendered HTML easily indexed and tracked).
  - Fast initial load for simple pages (server sends ready-to-display HTML).
  - Simple and predictable architecture (easy to implement and debug).
  - Works consistently across browsers (minimal JS dependency, graceful degradation).
  - Well-suited for large-scale, content-heavy applications with many discrete pages (examples: portals, e-commerce sites, knowledge bases).
- Limitations / Trade-offs:
  - Slower navigation (full-page reloads, repeated server round trips).
  - Limited interactivity without extra JavaScript.
  - Bad user experience compared to modern apps (interface feels less responsive and interactive due to network latency).
  - Higher server load from frequent full-page rendering.
  - Poor offline support (dependent on server availability).

Single-Page Applications (SPA)

- Structure:
  - Single page, content dynamically updated without full reloads
- Navigation Style:
  - Navigation and interactions update only relevant sections of the page
  - Browser URL may change using client-side routing
- Client-Server Interaction:
  - Client handles most UI rendering and partial business logic
  - Server primarily provides APIs or data responses
- Rendering:
  - Primarily client-side rendering (CSR)
  - Initial load may rely on SSR or pre-rendering to speed up initial content display.
- Examples:
  - Apps: Gmail, Trello, Slack Web, Google Maps
  - Frameworks: React, Vue.js, Angular, Svelte, SolidJS

Single-Page Applications (SPA)

- Key Features / Benefits:
  - Seamless, modern app interactivity (interface feels responsive; content updates dynamically and without noticeable delay).
  - Reduced network usage after initial load (only data/API requests are sent).
  - Complex workflows and rich interactivity (via client-side rendering and state management).
  - Frontend-backend decoupling enforces clear separation of concerns, improving project structure and maintainability (server handles APIs, client handles UI and interactivity).
  - Offline capability (cached data allows use without a continuous server connection).
  - Well-suited for dynamic, interactive, modern apps (e.g., Gmail, Trello, Google Maps)
- Limitations / Trade-offs:
  - Slower initial page load (large JavaScript bundles and rendering overhead)
  - Heavier client resource usage (more CPU/RAM needed to run complex apps).
  - SEO and analytics challenges (when server-side rendering or pre-rendering is not used).
  - Client-side development complexity (managing UI, routing, state updates and interactivity)

Hybrid / Mixed Applications

- Structure:
  - Multiple pages with partial dynamic updates within pages
- Navigation Style:
  - Main navigation may trigger full-page loads, while components update dynamically
  - Combines traditional page navigation with SPA-like interactivity
- Client-Server Interaction:
  - Server renders full pages and handles core business logic
  - Client updates interactive portions of pages dynamically
- Rendering:
  - Combination of server-side rendering (SSR) and client-side rendering (CSR)
  - Balances fast initial load with responsive interactivity
- Examples:
  - Apps: YouTube Web, Twitter Web, modern news sites
  - Frameworks: Next.js, Nuxt.js, Laravel Livewire, Remix

Hybrid / Mixed Applications

- Key Features / Benefits:
  - Moderate modern app interactivity (dynamic components update content; smoother than MPA, but navigation is less fluid than SPA).
  - Balanced network usage (dynamic components fetch only data/API, but full-page navigation still transfers the page content along with JavaScript).
  - Balanced initial load (server-rendered pages with dynamic components; faster than SPA, slower than MPA).
  - Improved SEO and analytics compared to SPA (server-rendered content enhances indexing and tracking; dynamic components may require extra handling).
  - Well-suited for content-heavy apps needing interactivity (e.g., YouTube Web, news sites).
- Limitations / Trade-offs:
  - Slower navigation than SPA (full-page reloads make some transitions less fluid).
  - High development and architectural complexity (mixes SSR and CSR).
  - Higher client resource usage than MPA (dynamic components increase CPU/RAM).
  - Offline use nearly impossible (most content and components require a server).

Rendering Strategies

Rendering Strategies in Web Applications

- What are Rendering Strategies?
  - Define how and where HTML is generated: server, client, or both.
  - Control when and how the UI updates, affecting performance and interactivity.
- Why They Matter:
  - Influence user experience: load speed and responsiveness.
  - Guide developer choices: framework, SEO, and state management.
  - Affect scalability: server load and bandwidth use.
- Relation to Web Application Models:
  - MPA: fits SSR and HATEOAS naturally.
  - SPA: often CSR, partial hydration, or resumability.
  - Hybrid: combines multiple strategies for best performance.

Fundamental Rendering Strategies

1. Server-Side Rendering (SSR)
   - HTML generated on the server per request.
   - Pros: fast initial page load, SEO-friendly.
   - Cons: higher server load, slower interactivity after the page is displayed.
   - Typical Frameworks: Laravel, Django, Rails.
2. Client-Side Rendering (CSR)
   - JavaScript renders and updates views in the browser dynamically.
   - Typically uses a Virtual DOM to efficiently update the UI (React, Vue, Angular), though some frameworks like Svelte compile components to optimized JS that updates the DOM directly.
   - Pros: smooth SPA interactivity, rich client-side experiences.
   - Cons: slower initial load, SEO requires additional handling.
   - Typical Frameworks: Vue.js, React, Angular, Svelte
3. Static Site Generation (SSG)
   - Pages are pre-rendered as static HTML files during the build process and served directly to the client.
   - Pros: very fast, scalable, SEO-friendly.
   - Cons: not suitable for frequently changing content.
   - Typical Frameworks: Hugo, Eleventy, Gatsby, Jekyll.

Modern / Advanced Rendering Strategies

4. Incremental Static Regeneration (ISR / Partial SSG)
   - Regenerate specific pages or fragments after the initial static build.
   - Pros: combines static performance with dynamic updates.
   - Cons: adds complexity; regeneration can introduce slight latency on first request after update
   - Typical Frameworks: Next.js.
5. Partial Hydration
   - Only selected parts of a fully server-rendered page are hydrated with JavaScript.
   - Pros: reduces JS execution and improves performance while keeping SSR benefits.
   - Cons: more complex to implement; requires careful component boundary design.
   - Typical Frameworks: Laravel Livewire, React Server Components
6. Islands Architecture
   - Mostly static HTML page with small, independent “islands” of interactive components.
   - Pros: minimal JS sent to client; interactivity isolated to islands.
   - Cons: can increase complexity in coordinating data and state between islands.
   - Typical Frameworks: Astro.
7. Resumability
   - Server-side application state is serialized and resumed on the client, avoiding full rehydration.
   - Pros: instant interactivity, minimal JS load.
   - Cons: increased complexity from managing serialized state.
   - Typical Frameworks: Qwik.

Lightweight & Emerging Strategies

9. HATEOAS / Hypermedia-Driven Rendering
   - Application state and navigation are encoded in hypermedia (DOM + HTTP verbs).
   - Pros: lightweight interactivity, minimal JavaScript, works naturally with MPA.
   - Cons: limited client-side interactivity; more server round-trips may be required for dynamic updates.
   - Typical Frameworks: HTMX.
10. Streaming / Progressive Rendering
   - HTML is sent in chunks as it becomes ready, improving perceived load time.
   - Pros: faster first content display, especially for large pages.
   - Cons: complex server logic; partial content errors can impact user experience.
   - Typical Frameworks: React 18 streaming SSR, Node.js streaming APIs
11. Multi-Strategy Rendering
   - Many modern frameworks combine multiple rendering strategies within the same application to optimize performance, interactivity, and scalability. For example, a page may use SSR for initial content, CSR for dynamic widgets, and partial hydration for selected components.
   - Pros: flexible; takes advantage of each strategy where it is most effective.
   - Cons: increased complexity; managing state and interactions across strategies can be challenging.
   - Typical Frameworks: Next.js, Nuxt.js, Qwik, Astro

SPA Frameworks

Characterization and examples

What Are SPA Frameworks

- Definition:
  - SPA (Single Page Application) frameworks are client-side JavaScript frameworks or libraries designed to build applications that run mostly in the browser, dynamically updating content without full page reloads.
- Core Idea:
  - Instead of the server sending new HTML for each navigation, the framework manages UI rendering, routing, and state within the client, communicating with the server mainly via APIs.
- Purpose:
  - Provide structure, reusable components, and efficient state handling for complex browser-based applications.
- Typical Features:
  - Component-based architecture
  - Reactive data binding
  - Virtual DOM or compiled rendering
  - Built-in routing and state management
  - Integration with SSR/SSG tools

Categories of SPA Frameworks

- Micro-Frameworks (Minimalist)
  - Provide minimal structure or enhancement for interactivity, without abstracting the DOM or managing app state.
  - Examples: Alpine.js, Stimulus, Lit
- UI Libraries (Lightweight)
  - Handle only the view layer; rely on external libraries for routing/state.
  - Examples: React, Preact, SolidJS
- Component Framework
  - Define reactive component model; routing/state optional
  - Examples: Vue.js, Svelte
- Complete SPA (opinionated) Frameworks
  - Include built-in routing, state, dependency injection, and tooling. Enforce strong conventions and architectural structure.
  - Examples: Angular, Ember,js
- Meta-Frameworks
  - Extend SPA frameworks with SSR, SSG, ISR capabilities.
  - Examples: Next.js (React), Nuxt.js (Vue), SvelteKit (Svelte)

Comparative Overview of Leading SPA Frameworks

Criteria	React	Vue.js	Angular	Svelte
Type	UI Library	Component Framework	Complete SPA Framework	Component Framework
Syntax/Language	JSX / JavaScript	Templates + JavaScript	TypeScript	HTML + JavaScript
Key Strengths	Modular, large ecosystem, component-driven	Simple, flexible, fast learning curve	Enterprise-grade, opinionated, dependency injection	Compiles to optimized JS, minimal runtime
Community / Adoption	Very High	High	High	Growing
Learning Curve	Medium	Easy	Steep	Easy
Performance	High	High	Moderate	Very High
Ecosystem Size	Very Large	Large	Large	Growing
SSR/SSG Support	Next.js, React Server Components	Nuxt.js	Angular Universal	SvelteKit
Best Fit	Modular, scalable apps	Balanced & educational	Enterprise systems	Small, performant SPAs

Why We Use Vue.js in This Course

- Simplicity: Easy to learn and understand; minimal boilerplate; clear syntax for beginners.
- Modern SPA Principles: Illustrates key SPA concepts such as client-side rendering, component composition, and reactive state.
- Balanced Complexity: Combines React's flexibility with Angular-like structure, making it ideal for learning SPA architecture.
- Component-Based Architecture: Clear separation of template, logic, and styles; promotes clean, maintainable code.
- Reactive Data Model: Simple but powerful reactivity system for dynamic UI.
- Optional Routing & State Management: Can scale with official libraries (Vue Router, Pinia) without imposing them.
- Tooling & Ecosystem: Vite-powered CLI, single-file components, strong community support.

Mobile and Multiplatform

Cross-Platform Applications

Native and Cross-Platform Applications

- As web development evolves, the line between web and native applications is increasingly blurred. The Single-Page Application (SPA) model, which centralizes logic and relies on APIs, enables cross-platform development and the reuse of a core codebase across browsers and mobile devices.

- Mobile development encompasses native, native-like, hybrid, and web-based approaches, each optimized for different performance, feature access, development complexity, and market needs.

- Web technologies and SPA are particularly valuable for web-based and hybrid apps, allowing developers to share code across platforms, increase productivity, and simplify maintenance. Their primary goal is efficient development and faster deployment, rather than matching native performance, making them suitable for applications where high performance is not required.

Architectural Models for Multiplatform Development

Model		Description	Platform Specific Code	Suited for	Tech Stack
Native Applications		Applications built specifically for a platform (iOS/Android) using native languages (Swift, Kotlin, Java); code is not shared across platforms.	High. Full access to all native APIs and UI components.	High-performance apps; apps requiring full access to platform-specific features; platform-optimized consumer apps	Swift (iOS), Kotlin, Java, …
Cross-Platform	PWA (Progressive Web App)	A standard web application that runs in any modern browser, enhanced with APIs that enable offline use and installation on a device without relying on an app store.	Zero. It's a web app; relies solely on web standards and modern browser features.	Web-first apps with offline support; ideal for content-heavy applications; easy distribution without app stores	HTML, CSS, JavaScript, React, Vue.js, Angular, Nuxt PWA, Quasar, Workbox, …
	Hybrid (WebView Based)	A web application built with HTML, CSS, and JavaScript, packaged inside a native container that runs the web interface through a WebView and uses native bridges to access device APIs.	Low. Only the native bridges code; logic is entirely web code.	Web apps needing device features; apps that reuse web code across platforms; moderate-performance consumer apps	HTML, CSS, JavaScript, Ionic, Cordova, Capacitor, Framework7, …
	Native-Like Cross-Platform	Code (e.g., JavaScript or Dart) renders native UI components or uses its own UI engine, providing a native-like experience instead of a web page.	Moderate. Shared business logic, but UI components are translated to native elements or custom-rendered	High-performance, interactive apps; apps requiring deep native integration; large-scale consumer apps	React Native, Flutter, Xamarin, Kotlin Multiplatform, Dart, …

PWA – Progressive Web App

- Web app running in any modern browser, designed as a SPA.
- Enhanced with Service Workers (offline caching), Web App Manifest (installability), and optional push notifications.
- Uses only web technologies — including localStorage, IndexedDB, Cache API, Fetch API, WebSockets, Background Sync, Geolocation API, and others; no native code required.
- Can be accessed immediately via a browser and added to a device's home screen as an app icon via the Web App Manifest, giving the appearance of a native app without using the app store.

Turning a SPA into a PWA

- Create the SPA with responsive design
  - Ensure the app layout adjusts to mobile screen sizes.
- Add a Web App Manifest
  - Defines app name, icons, theme color, and start URL.
  - Enables “add to home screen” functionality.
- Register Service Workers (recommended)
  - Enables offline caching, background sync, and updates.
  - Optional for minimal install, needed for offline and advanced features.
- Enable HTTPS Deployment
  - Required for Service Workers and secure features.
- Optional Features
  - Push notifications, background sync, WebSockets.
- Test PWA Capabilities
  - Verify offline behavior, add-to-home-screen, caching.

Vue.js specific tools / frameworks:

Manifest & Service Workers:
@vue/cli-plugin-pwa, Nuxt PWA module, vue-pwa-plugin

Push notifications / advanced caching:
Workbox, Firebase Cloud Messaging (FCM)

Quasar Framework

Hybrid (WebView Based)

- Web application built with HTML, CSS, and JavaScript, packaged inside a native container that runs the web interface through a WebView.
- Relies on a hybrid framework or runtime layer, such as Capacitor or Cordova, to provide the container and bridge communication between web code and native APIs like camera, GPS, storage, and notifications.
- Most of the logic remains in the web layer, while a small native layer manages the WebView and exposes device capabilities through JavaScript interfaces.
- Supports offline caching and client-side storage using standard web technologies (localStorage, IndexedDB, Cache API).
- Runs on iOS and Android using the same web codebase, allowing fast cross-platform deployment with optional integration of native features like background tasks or push notifications.

Considerations for Developing a SPA as a Hybrid App

- Create the SPA with responsive design
  - Ensure the app layout adjusts to mobile screen sizes.
- Wrap the SPA using a hybrid framework or runtime layer
  - Use a WebView within the framework (e.g., Capacitor or Cordova) to render the web interface inside a native container.
- Add native bridges through the hybrid framework
  - Expose device APIs (camera, GPS, storage, notifications) to the web layer via the framework's bridge system.
  - Requires minimal native code — communication between web and native layers is managed by the framework.
- Optional features through native plugins
  - Enable push notifications, background tasks, and deeper native integrations using the framework's plugin system.
- Test hybrid app capabilities
  - Verify device API access, offline caching (if implemented), and WebView rendering performance across iOS and Android.

Vue.js specific tools / frameworks:

Hybrid framework / runtime:
Capacitor, Cordova, Ionic, Framework7

Push / advanced features:
Capacitor Push API,
Firebase Cloud Messaging (FCM)

Quasar Framework

- MDN Web Docs, “HTTP: Hypertext Transfer Protocol” https://developer.mozilla.org/en-US/docs/Web/HTTP
- MDN Web Docs, “HTTP/3” https://developer.mozilla.org/en-US/docs/Glossary/HTTP_3
- web.dev (Chrome), “Rendering on the Web” https://web.dev/articles/rendering-on-the-web
- Next.js, “How to implement Incremental Static Regeneration (ISR)” https://nextjs.org/docs/app/guides/incremental-static-regeneration
- Astro Docs, “Islands architecture” https://docs.astro.build/en/concepts/islands
- Qwik Docs, “Resumable (Concepts)” https://qwik.dev/docs/concepts/resumable
- React Docs, “Server Components” https://react.dev/reference/rsc/server-components
- htmx, “Documentation” https://htmx.org/docs
- MDN Web Docs, “Progressive Web Apps (PWA)” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- MDN Web Docs, “Web application manifest” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Manifest

- MDN Web Docs, “Service Worker API” https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- Chrome for Developers, “Workbox (overview)” https://developer.chrome.com/docs/workbox
- Vue.js, “Guide — Introduction (Vue 3)” https://vuejs.org/guide/introduction
- React, “Quick Start (Learn)” https://react.dev/learn
- Angular, “What is Angular?” https://angular.dev/overview
- Svelte, “Interactive Tutorial” https://svelte.dev/tutorial
- Capacitor, “Docs — Cross-platform Native Runtime for Web Apps” https://capacitorjs.com/docs/
- Apache Cordova, “Documentation” https://cordova.apache.org/docs/en/12.x
- React Native, “Introduction / Getting Started” https://reactnative.dev/docs/getting-started
- Flutter, “Docs — Get started with Flutter” https://docs.flutter.dev
- "State of JS Survey", https://stateofjs.com (2024)
- "2025 Developer Survey" https://survey.stackoverflow.co/2025 (2025)

Acknowledge

The authors acknowledge the use of ChatGPT (https://chatgpt.com/) and Google Gemini (https://gemini.google.com) as a tool supporting content development and text composition. Full responsibility for the final content rests with the authors.